RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

Urban Air Quality Intelligence System with Predictive Analytics

MLOps Tutorial Report

submitted by

L MORYAKANTHA	**1RV24AI506**
VINEET RAJ	**1RV23AI132**
MAHANTESH PB	**1RV24AI407**
ABHINAV P	**1RV23AI135**

Artificial Intelligence and Machine Learning

2024-2025

RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

DECLARATION

We, **L Moryakantha, Vineet Raj, Mahantesh PB and Abhinav P**, the students of the fourth semester B.E., Department of Artificial Intelligence and Machine Learning, RV College of Engineering, Bengaluru-560059, bearing USN: **1RV24AI406, 1RV23AI132, 1RV24AI407 and 1RV23AI135** hereby declare that the EL topic titled **'Urban Air Quality Intelligence System with Predictive Analytics'** has been carried out by us and submitted in partial fulfillment of the coursework requirements for the award of Degree in Bachelor of Engineering in **Artificial Intelligence and Machine Learning** of the **Visvesvaraya Technological University, Belagavi** during the year **2025-2026**.

Further, we declare that the content has not been submitted previously by anybody for the award of any Degree or Diploma to any other University.

We also declare that any intellectual property rights generated from this project at RVCE will be the property of RV College of Engineering, Bengaluru, and we will be among the authors.

Place: Bangalore

Date: 05/01/2026 **Signature**

L Moryakantha _____

Vineet Raj _____

Mahantesh PB _____

Abhinav P _____

ACKNOWLEDGEMENT

We express our sincere gratitude to RV College of Engineering, Bengaluru, for providing us with the opportunity and resources to carry out this Experiential Learning (EL) project.

We would like to extend our heartfelt thanks to the Department of Artificial Intelligence and Machine Learning for their continuous support and encouragement throughout the course of this work. We are especially grateful to our EL project guide(s) and faculty members for their valuable guidance, constructive feedback, and constant motivation, which played a crucial role in the successful completion of this project.

We also thank the college administration and supporting staff for providing a conducive environment and necessary facilities for carrying out this project.

Finally, we would like to thank our friends and peers for their cooperation and support during the course of this work.

ABSTRACT

Rapid urbanization and industrial growth in developing nations have led to a significant deterioration in air quality, posing severe health risks to densely populated communities. Fine particulate matter (PM2.5) serves as a critical indicator of pollution, yet accurate forecasting remains a challenge due to the complex, non-linear interactions between atmospheric conditions and anthropogenic factors. Traditional machine learning approaches often fail in real-world deployment due to "data drift," where shifting environmental patterns degrade model performance over time. Addressing these challenges requires not only robust predictive modeling but also a sustainable operational framework that ensures reliability, continuous monitoring, and scalability in production environments.

This project presents an end-to-end Urban Air Quality Intelligence System, designed to predict daily PM2.5 concentrations using a comprehensive historical dataset from air quality monitoring stations across India. The system integrates data preprocessing, feature engineering, and model training within an automated pipeline. The dataset encompasses key pollutants—including PM10, Nitrogen Dioxide (NO_2), Sulfur Dioxide (SO_2) and Carbon Monoxide (CO)—alongside engineered temporal features to capture seasonal trends and daily fluctuations. The primary objective is to provide actionable insights for public health safety and policy-making by strictly modeling the target variable, PM2.5, through advanced regression techniques.

To achieve high predictive accuracy, three distinct machine learning models were implemented and rigorously evaluated: Linear Regression to establish a baseline for linear dependencies, Random Forest Regressor to handle non-linear data structures, and XGBoost Regressor for its superior ability to model complex feature interactions. After extensive hyperparameter tuning, XGBoost was selected as the production model, demonstrating the best performance with the lowest Root Mean Squared Error (RMSE). The evaluation phase utilized standard regression metrics, including Mean Absolute Error (MAE) and R-squared (R^2) scores, ensuring the chosen model generalizes well to unseen data rather than merely memorizing the training set.

Moving beyond static modeling, the system incorporates a comprehensive MLOps (Machine Learning Operations) framework to guarantee reproducibility and governance. The pipeline is orchestrated using Argo Workflow with Kubernetes, with Docker ensuring consistent environments across development and production. Experiment tracking and model versioning are managed via Weights & Biases (W&B) and CometML, allowing for real-time visualization of training metrics. For deployment, the model serves as a REST API using FastAPI, monitored by Prometheus for system latency and errors, while AIF360 is employed to detect and mitigate algorithmic bias. This integration of state-of-the-art tools ensures the system remains robust, fair, and maintainable, bridging the gap.

GLOSSARY

A. **AIF360:** A toolkit used for fairness, bias, and governance analysis to ensure model predictions are not biased across different regions or time periods.
B. **Central Pollution Control Board (CPCB):** The authoritative repository from which the project's air quality data (including city-level aggregates) was collected.
C. **CI/CD (GitHub Actions):** Continuous Integration and Continuous Deployment practice used to automatically build Docker images, run tests, and deploy the updated API whenever code is pushed.
D. **CometML:** A tool used alongside Weights & Biases for experiment tracking, model versioning, and logging training metrics.
E. **Concept Drift:** A specific type of model degradation where the statistical properties of the target variable change over time, necessitating continuous monitoring.
F. **Data Drift:** The phenomenon where input data patterns change over time (e.g., due to seasonal or environmental shifts), causing models to underperform in production.
G. **Docker:** A platform used for containerization that packages training and inference environments into images to ensure consistency across different systems.
H. **FastAPI:** A modern web framework used to create the REST API (specifically the `/predict` endpoint) for real-time model inference.
I. **Feature Engineering:** The process of transforming raw data into useful input signals, such as extracting hour, day, and month from timestamps or creating rolling averages.
J. **Grafana:** A visualization tool used to create dashboards for monitoring stored metrics and data trends.
K. **Hyperparameters:** Configuration settings for the models (e.g., learning rate, tree depth) that are tuned and logged to optimize performance.
L. **Argo Workflows:** A Kubernetes-native pipeline orchestration tool that defines and automatically executes workflow steps such as data preprocessing, model training, evaluation, and monitoring using DAG-based execution.
M. **Linear Regression:** A baseline machine learning model used to understand simple linear relationships between pollutants and PM2.5 levels.
N. **MLOps (Machine Learning Operations):** A set of practices integrating ML development with deployment, monitoring, and governance to ensure scalable and reliable AI systems.
O. **Pandas & NumPy:** Core Python libraries used for data cleaning, preprocessing, missing value imputation, and numerical analysis.
P. **PM2.5:** Fine particulate matter (2.5 micrometers or smaller); the primary target variable for the prediction model due to its critical impact on public health.
Q. **Prometheus:** A monitoring tool that tracks system performance metrics such as API request counts, latency, and error rates.

R. **Random Forest Regressor:** An ensemble learning method that handles non-linear relationships and is robust against noise and overfitting.
S. **RMSE (Root Mean Squared Error):** The primary evaluation metric used to select the best model; it penalizes large errors and is ideal for continuous data like pollution levels.
T. **R^2 Score (Coefficient of Determination):** A statistical measure used to evaluate how well the model explains the variance in the PM2.5 data.
U. **Scikit-learn:** A machine learning library used for training models, evaluating performance, and building preprocessing pipelines.
V. **Weights & Biases (W&B):** A platform for experiment tracking that logs metrics like RMSE, enables visual model comparison, and manages model artifacts.
W. **XGBoost Regressor:** A gradient boosting-based ensemble model selected as the best-performing algorithm for its ability to capture complex feature interactions with the lowest error.

TABLE OF CONTENTS

CHAPTER 1

1.1. Problem Statement and Objectives

1.1.1 Problem Statement

The problem addressed in this project is the design and implementation of an **Urban Air Quality Intelligence System with Predictive Analytics** that leverages machine learning and MLOps practices to deliver reliable, scalable, and interpretable **PM2.5** predictions.

1.1.2 Objectives

1. To design and develop a robust machine learning model for predicting PM2.5 concentration levels using historical air quality, meteorological, and temporal data.
2. To evaluate multiple machine learning algorithms and select the best-performing model based on standardized evaluation metrics.
3. To deploy the trained model as a real-time prediction service using a scalable and containerized architecture.
4. To establish an end-to-end MLOps pipeline for automated model training, validation, deployment, and monitoring.

1.2 Project Scope and Evaluation Metrics

1.2.1 Project Scope.

In Scope

- **Prediction:** Estimating PM2.5 levels using input features including PM10, Ozone (O3), Carbon Monoxide (CO), and temporal features (hour, day, month).
- **Inference:** Supporting both batch and real-time inference via a FastAPI application.
- **Orchestration:** Automating workflows (ingestion, preprocessing, training, evaluation, drift checks) using Argo Workflows with Kubernetes.
- **Monitoring:** Continuous tracking of API performance and data drift using Prometheus, Grafana, and statistical tests.
- **Governance:** Versioning models, tracking experiments, and maintaining documentation such as Model Cards and Audit Checklists.

Out of Scope

- **Forecasting:** Multi-step forecasting of future PM2.5 trajectories (time-series forecasting).
- **Causal Inference:** Explaining the specific sources or causes of pollution.
- **Broad Deployment:** Multi-city production rollout beyond the specific Indian cities provided in the dataset.

1.2.2 Evaluation Metrics

Model Performance Metrics

- **RMSE (Root Mean Squared Error):** The primary metric used to penalize large errors and avoid extreme inaccuracies.
- **MAE (Mean Absolute Error):** Used to provide an interpretability baseline for average error.
- **R^2 (Coefficient of Determination):** Measures the proportion of variance in the dependent variable that is explained by the model.
- **MAPE (Mean Absolute Percentage Error):** Provides a relative error measure, useful for understanding performance in a business context.
- **SHAP Values (SHapley Additive exPlanations):** Used to quantify the contribution of each feature (e.g., PM10, CO) to the model's predictions, providing both local (single prediction) and global (overall model behavior) explainability.

Operational Metrics

- **API Latency (P95):** Tracked via Prometheus to ensure the system remains responsive under load.
- **API Uptime and Error Rate:** Tracked via Prometheus to guarantee high availability of the service and percentage of failed requests.
- **Drift Detection:** Custom Kolmogorov-Smirnov (KS) and Population Stability Index (PSI) tests are run to detect instability and trigger retraining.

Fairness Metrics

- **Disparate Impact and Statistical Parity:** Calculated using AIF360 to ensure outcome parity across different groups and measure the independence of the prediction from protected attributes.

1.3 Tools and Environmental Setup

1.3.1. Development and Execution Environment

- **Operating System:** Windows 10 / Ubuntu Linux
- **Programming Language:** Python (≥ 3.9)

- **IDE & Notebooks:** Visual Studio Code, Jupyter Notebook
- **Environment Management:** Conda / Virtual Environment (venv)

1.3.2 Data Engineering and Feature Processing Tools

Tool	Role in MLOps Pipeline
Pandas	Data ingestion, cleaning, and transformation
NumPy	Numerical computations
Scikit-learn Pipelines	Feature scaling, encoding, and preprocessing

1.3.3 Machine Learning and Model Development Tools

Tool	Role in MLOps Pipeline
Scikit-learn	Baseline models, preprocessing, evaluation
XGBoost	High-performance regression for PM2.5 prediction
Joblib / Pickle	Model serialization

1.3.4. Experiment Tracking and Model Management

Tool	Role in MLOps Pipeline
CometML Tracking	Logging parameters, metrics, and artifacts
Git	Source code version control

1.3.5 Model Packaging and Deployment Tools

Tool	Role in MLOps Pipeline
FastAPI	REST API for model inference as backend serving
Docker	Containerization of model and dependencies

1.3.6 CI/CD and Automation Tools

Tool	Role in MLOps Pipeline
GitHub Actions	Automated training, testing, and deployment

1.3.7 Monitoring, Drift Detection, and Observability Tools

Tool	Role in MLOps Pipeline
Prometheus	System and API performance metrics
Grafana	Visualization dashboards
Custom Drift Checks	PSI and KS-test based monitoring

1.3.8 Governance and Documentation Tools

Tool	Role in MLOps Pipeline
CometML Model Registry	Model lifecycle governance
Model Cards	Documentation of model behavior
Audit Logs	Traceability and compliance

1.3.9 Hardware and System Requirements

Component	Specification
Processor	Intel i5 / AMD Ryzen 5 or higher
RAM	Minimum 8 GB (16 GB recommended)
Storage	50 GB free disk space
Network	Stable internet connection of >=1 MB/s

1.4 Literature Review -

Title	Author(s)	Year	Summary	DOI Link
Air Quality Prediction Using Machine Learning Algorithms: A Review	Madan et al.	2020	A comprehensive review of standard ML algorithms (SVM, Random Forest) applied to pollution datasets, providing a baseline for model selection.	10.1109/ICACCCN51052.2020.9362912
PM2.5 prediction based on random forest, XGBoost, and deep learning	Joharestani et al.	2019	Demonstrates that XGBoost often outperforms traditional regression models by effectively capturing non-linear feature interactions in multi-source data.	10.1038/s41598-019-45784-y

Air pollution prediction using deep learning models	Khan et al.	2020	Compares LSTM and RNN performance for air quality time-series forecasting, highlighting the superiority of deep learning for temporal dependencies.	10.1007/s11356-019-07427-5
PM2.5-GNN: A Domain Knowledge Enhanced Graph Neural Network	Wang et al.	2020	Introduces a Graph Neural Network approach to capture spatial dependencies between monitoring stations, improving forecasting accuracy.	10.1145/3397536.3422208
Air quality prediction using explainable AI (SHAP)	Strydom et al.	2021	Applies SHAP to interpret model predictions, bridging the gap between high-accuracy black-box models and transparency.	10.1109/ACCESS.2021.3137782

Edge MLOps: An Automation Framework for AIoT Applications	Raj et al.	2021	Proposes a framework for deploying and maintaining ML models on edge devices, addressing operational challenges.	10.1109/LCN52139.2021.9524993
A Machine Learning Model for Air Quality Prediction for Smart Cities	Mahalingam et al.	2019	Discusses the integration of ML prediction models into smart city infrastructure for real-time urban planning and health alerts.	10.1109/WiSPNET45539.2019.9032734
Federated Learning for Privacy-Preserving Air Quality Forecasting	Yi et al.	2023	Presents a federated learning framework that allows training on distributed sensor data without sharing raw data, preserving privacy.	10.1016/j.iot.2023.100781
Prediction of Air Quality Using LSTM Recurrent Neural Network	Raheja et al.	2022	Evaluates the efficiency of LSTM networks specifically for the prediction of multiple pollutants in urban environments.	10.4018/IJSI.297982

Graph Neural Network for Air Quality Prediction	Iskandaryan et al.	2023	Demonstrates the use of Attention Temporal Graph Convolutional Networks (A3T-GCN) to model complex spatiotemporal traffic and pollution data.	10.1109/ACCESS.2023.3234214
Machine Learning-Based Forecasting of Air Quality Index (Comparative)	Tırınk	2024	A recent study evaluating XGBoost, LightGBM, and SVM, confirming that ensemble tree-based models generally yield the lowest RMSE.	10.1371/journal.pone.0334252
PM2.5 Concentration Prediction Model Based on Random Forest and SHAP	Zhang et al.	2024	Combines Random Forest with SHAP analysis to not only predict PM2.5 levels but also quantify the contribution of meteorological features.	10.1142/S0218001424520128

Prediction of PM2.5 Concentration Using Spatiotemporal Data	Ma et al.	2023	Focuses on incorporating spatiotemporal features into ML models to handle the dynamic nature of air pollution dispersion.	10.3390/atmos14101517
Predicting PM2.5 Using Random Forest Enhanced with Polynomial Features	Goyal et al.	2024	Investigates enhancing Random Forest models with polynomial feature engineering to better capture non-linear relationships.	10.1109/IS61756.2024.10705219
Machine Learning Algorithms for Air Pollutants Forecasting	Bhatnagar et al.	2020	A foundational conference paper comparing multiple standard algorithms (Linear Regression, Decision Trees) for initial baselines.	10.1109/SIITME50350.2020.9292238

Global Calibration Models Match Ad-Hoc Calibrations (Concept Drift)	De Vito et al.	2022	Addresses "Concept Drift" in low-cost sensor networks, proposing calibration models that adapt to changing environmental conditions.	10.1109/ISOEN54820.2022.9789669
Optimal Field Calibration of Multiple IoT Low Cost Air Quality Monitors	Esposito et al.	2020	Discusses the challenges of data reliability and drift in IoT sensor networks, a critical component of "MLOps for IoT".	10.1007/978-3-030-58814-4_57
Learning under Concept Drift: A Review	Lu et al.	2018	The foundational review on Concept Drift detection and adaptation, essential for maintaining long-term accuracy in environmental ML systems.	10.1109/TKDE.2018.2876857

Explainable AI for Urban Air Quality: SHAP Interpretation	Gowri et al.	2025	A cutting-edge study (published online 2024/2025) using SHAP to interpret stacked ensemble forecasts for urban air quality.	10.1007/s00704-025-05741-3
Environmental Justice and the Use of AI in Urban Air Pollution Monitoring	Whiteside/ Zou	2022	Explores the intersection of AI, monitoring, and environmental justice, addressing fairness and bias in pollution measurement.	10.1007/s40572-022-00346-y

CHAPTER 2

2.1 Understanding MLOps, Risks and Requirement Analysis

Machine Learning Operations (MLOps) refers to a set of practices that combine machine learning development with operational engineering to manage the complete lifecycle of machine learning systems. As a result, maintaining reliable performance in production requires continuous monitoring, automation, and governance.

In the **Urban Air Quality Intelligence System with Predictive Analytics**, the machine learning model predicts PM2.5 concentration levels using environmental and meteorological data. Since air quality data is dynamic, seasonal, and influenced by multiple external factors, the system is exposed to risks such as data quality issues, model drift, deployment failures, and misinterpretation of predictions. These characteristics make MLOps essential for ensuring long-term reliability and trustworthiness of the system.

2.1.1 Role of MLOps in the Proposed System

- Continuously ingest and validate environmental data from multiple sources
- Track experiments, parameters, and model versions systematically
- Deploy models in a reproducible and scalable manner
- Monitor model performance, data drift, and system health
- Trigger retraining and redeployment when performance degrades

2.1.2 Risks in Air Quality MLOps Systems

- **Data Risks:** Poor-quality or incomplete environmental data can directly degrade model accuracy.
- **Model Risks:** Seasonal variation and data drift can cause performance degradation after deployment.
- **Technical Risks:** Integration failures among MLOps tools such as data versioning, experiment tracking, and deployment frameworks can disrupt the pipeline.
- **Operational Risks:** Deployment failures, API downtime, or inadequate monitoring can affect system availability.
- **Governance and Ethical Risks:** Lack of reproducibility, insufficient documentation, or misinterpretation of predictions may reduce trust and accountability.

2.1.3 Requirement Analysis

Functional Requirements

- The system shall ingest historical and real-time air quality data.
- The system shall validate and preprocess incoming data.
- The system shall predict PM2.5 concentration levels accurately.
- The system shall expose predictions through a RESTful API.
- The system shall monitor model performance and data drift.
- The system shall support automated retraining and redeployment.

Non-Functional Requirements

- The system shall provide predictions with low latency.
- The system shall maintain high availability and reliability.
- The system shall ensure reproducibility of experiments.
- The system shall support version control for data and models.
- The system shall provide transparency through logging and documentation.

2.2 Risk Matrix

A **5 × 5 Risk Assessment Matrix** is used to systematically evaluate and prioritize risks associated with the **Urban Air Quality Intelligence System with Predictive Analytics**. The matrix evaluates risks based on two dimensions:

Probability and **Impact**, each measured on a scale of 1 to 5.

The overall **Risk Score** is calculated as: **Risk Score = Probability × Impact**

2.2.1 Risk Scales Definition

Probability Scale

Score	Description
1	Rare
2	Unlikely
3	Possible
4	Probable
5	Highly Probable

Impact Scale

Score	Description
1	Very Low

2	Low
3	Medium
4	High
5	Very High

2.2.2 Risk Severity Classification

Risk Score Range	**Severity Level**
1 – 3	Low Risk
4 – 6	Low–Moderate Risk
7 – 12	Moderate Risk
13 – 18	Moderate–High Risk
19 – 25	High / Critical Risk

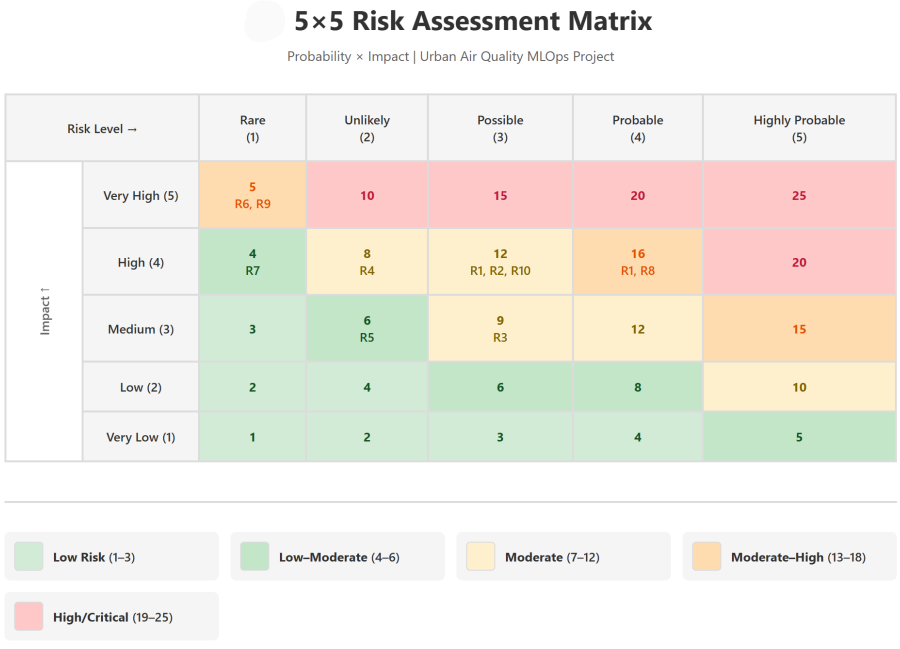


Figure1: Risk Matrix

2.2.3 Identified Project Risks and Scores

Risk Score Calculation: Probability (1–5) × Impact (1–5) = Risk Score (1–25)

Risk ID	Risk Description	Probability	Impact	Risk Score	Severity
R1	Data quality issues	Probable (4)	High (3)	12	Moderate
R2	Model drift due to seasonal variation	Possible (3)	High (3)	9	Moderate
R3	Integration issues among MLOps tools	Possible (3)	Medium (3)	9	Moderate
R4	Reproducibility failure	Unlikely (2)	High (4)	8	Moderate
R5	Deployment downtime	Possible (3)	Medium (2)	6	Low–Moderate
R6	Misinterpretation of forecasts	Unlikely (2)	Very High (5)	10	Major
R7	Data or model loss	Rare (1)	High (4)	4	Low–Moderate
R8	Poor model generalization	Probable (4)	Medium (2)	8	Moderate
R9	Legal or compliance risk	Rare (1)	Very High (5)	5	Low–Moderate
R10	Monitoring failure	Possible (3)	High (4)	12	Major

CHAPTER 3

3.1 Data Collection and Exploration Techniques

3.1.1 Data Collection

- **Sources:** The primary data comes from the "Kaggle India Air Quality" dataset (2015–2020) and "Indian Cities AQI" dataset (2020-2024).
- **Links to the datasets**: Kaggle India Air Quality | Indian Cities AQI
- **Volume and Coverage:** The raw data consists of approximately 14 million observations derived from 453 individual CSV files representing specific monitoring stations (e.g., Delhi, Bihar, Andhra Pradesh) and city-level aggregates.
- **Ingestion Process:** A dedicated script (scripts/data_ingestion.py) is used to load per-station CSVs, consolidate them into a unified format, and standardize timestamps.
- **Raw Features:** The dataset initially contained over 90 columns, including pollutants (PM10, NO2, SO2, CO, O3, NH3) and meteorological variables (Temperature, Humidity, Wind Speed).
- **Missing Values:** Significant missingness was detected, with many feature columns missing 30–60% of their data. The target variable (PM2.5) was missing in approximately 15% of records.
- **Inconsistencies:** Major quality issues included duplicate columns recording the same metric in different units and extreme outliers (e.g., sensor errors producing values like 999 µg/m³).

3.1.2 Exploratory Data Analysis (EDA)

Statistical Correlations analysis identified which co-pollutants were strong proxies for PM2.5:

- **PM10**: Strongest positive correlation (0.87), making it the primary predictor.
- **CO** (Carbon Monoxide): Moderate correlation (0.42), serving as a signal for industrial/traffic activity.
- **O3** (Ozone): Weak negative correlation (-0.15), retained for chemical completeness.
- **Exclusions**: Meteorological variables (**Temp**, **Humidity**) were excluded from the final model due to high **missingness** (>50%), despite their theoretical relevance.

3.2 Data Profiling, Cleaning and Preprocessing

3.2.1 Data Profiling

- Percentage of missing values per feature

- Detection of duplicate columns representing the same pollutant
- Identification of invalid sensor readings
- Temporal continuity checks across timestamps

Features with more than **50% missing values** were flagged for exclusion, while the target variable **PM2.5**, with approximately **15% missing values**, was retained for further processing.

3.2.2 Data Cleaning

The data cleaning process involved multiple corrective steps:

- **Missing Value Handling**
  - PM2.5 missing values were removed to avoid target leakage.
  - Predictor variables with missing values below 30% were imputed using median-based imputation.
  - Features exceeding the missing value threshold were dropped
- **Outlier Treatment**
  - Extreme pollutant values caused by sensor malfunction (e.g., PM2.5 > 900 $\mu g/m^3$) were removed.
  - Interquartile Range (IQR) method was applied to filter unrealistic readings.
- **Unit Standardization**
  - Duplicate columns with different measurement units were normalized into a single consistent unit.
  - Redundant columns were removed to avoid multicollinearity.

3.2.3 Data Preprocessing

To prepare the dataset for machine learning:

- Numerical features were scaled using standard normalization.
- Timestamps were converted into structured temporal fields.
- Data was resampled to uniform time intervals to ensure temporal consistency.

3.3 Feature Extraction and Feature Selection

Feature Selection Criteria

- Strength of relationship with PM2.5
- Availability and completeness of data
- Interpretability of features
- Impact on training and inference time

Feature Selection Methods

- **Correlation analysis** to identify strong predictors of PM2.5
- **Model-based feature importance** using tree-based models
- **Domain knowledge** of pollutant interactions

Final Selected Features

- **PM10** – strongest predictor of PM2.5
- **CO** – moderate indicator of traffic and industrial activity
- **O3** – retained for contextual relevance
- Meteorological features were excluded due to high missingness and limited contribution.

3.4 Model Design Approaches

Model Selection Criteria

- RMSE and R^2 performance
- Training time < 30 minutes on CPU
- Inference latency < 100 ms
- Model interpretability

Data Splitting Strategy

- Temporal split without shuffling:
  - 60% Training
  - 20% Validation
  - 20% Testing

Models Used

- **Linear Regression** – baseline and highly interpretable
- **Random Forest Regressor** – handles non-linearity and noise
- **XGBoost Regressor** – best overall performance and efficiency

Hyperparameters were tuned using validation data, and experiments were tracked using **W&B / Comet**.

Model Acceptance Criteria

A model was approved for deployment only if:

- RMSE < 30 µg/m³
- MAE < 20 µg/m³
- $R^2 > 0.75$
- MAPE < 40%

3.5 Governance and Data Version Control

Version Control and Reproducibility

- **Git** for code version control

Experiment Governance

- All experiments, parameters, and metrics logged using **W&B / Comet**
- Only validated models were promoted for deployment

Fairness and Auditability

- Disparate Impact maintained within **0.8–1.25**
- Statistical Parity Difference < **0.1**
- Complete audit trails maintained for models and data

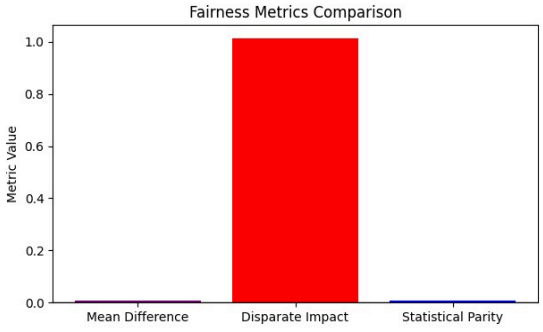


Figure : Disparity Impact (Goveranance)

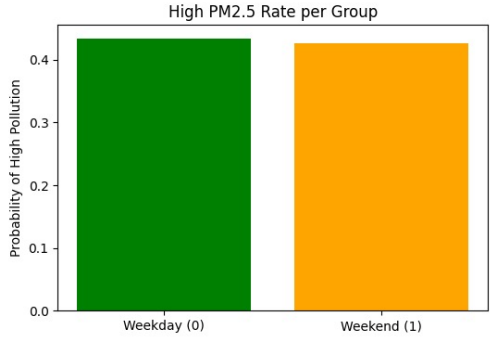
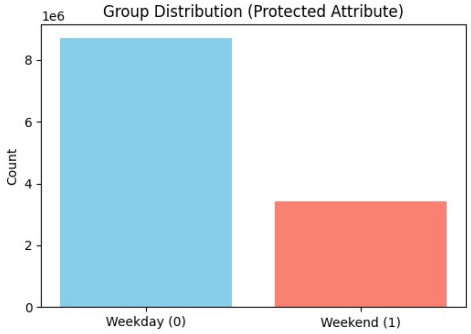


Figure : Probability of Pollution of Weekend v/s Weekday



CHAPTER 4

4.1 Model Building

In this phase, multiple machine learning models were developed to predict **PM2.5 concentration**, a continuous numerical value indicating air pollution severity.

Steps followed:

- Selected **PM2.5 (µg/m³)** as the target variable.
- Chosen input features include pollutant levels (PM10, CO, NO_2, O_3), meteorological factors (temperature, humidity, wind speed), and time-based features (hour, day, month).
- Data was normalized and missing values were handled to ensure stable training.
- Three regression models were implemented:
    - **Linear Regression** – baseline model
    - **Random Forest Regressor** – ensemble model
    - **XGBoost Regressor** – gradient boosting model

These models were selected to compare linear, ensemble, and boosting-based approaches.

4.2 Training, Validation, and Testing

To ensure unbiased evaluation, the dataset was split into:

- 60% Training
- 20% Validation
- 20% Testing

Training process:

- Models were trained on the training dataset using optimized hyperparameters.
- Validation was done using test data to check generalization performance.
- Random seeds were fixed to ensure reproducibility.

4.3 Evaluation Metrics and Visualization

Since this is a **regression problem**, the following metrics were used:

Metrics:

- **RMSE (Root Mean Squared Error)** – measures average prediction error magnitude
- **MAE (Mean Absolute Error)** – measures average absolute deviation
- **R² Score** – indicates how well the model explains variance in data

Visualizations:

- Actual vs Predicted PM2.5 plots
- Residual distribution plots
- Feature importance plots (for tree-based models)

4.4 Experiment Tracking

To track experiments systematically, **Weights & Biases (W&B)** was used.

Tracking included:

- Model parameters and hyperparameters
- Training and testing RMSE values
- Comparison of model performances
- Best model selection and versioning
- Model artifacts stored for reuse

4.5 Interpreting Model Results and Error Analysis

Model Interpretation:

- **XGBoost** provided feature importance scores indicating which factors influence PM2.5 most basically **XGBoost is proved to be the best model.**
- Pollutants like PM10, CO, and meteorological factors showed high influence.

Error Analysis:

- Residuals were analyzed to identify high-error predictions.
- Larger errors were observed during extreme pollution events, indicating complex nonlinear behavior.
- This justified the use of ensemble and boosting models over simple linear models.

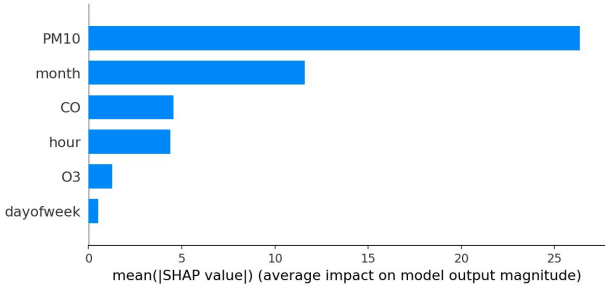


Figure : Interpretability and Selection of Important Attribute

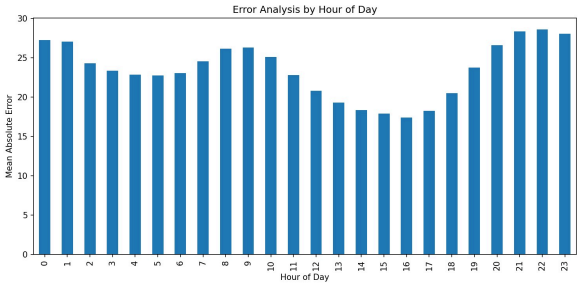


Figure : Error Analysis and Tracking Per Day

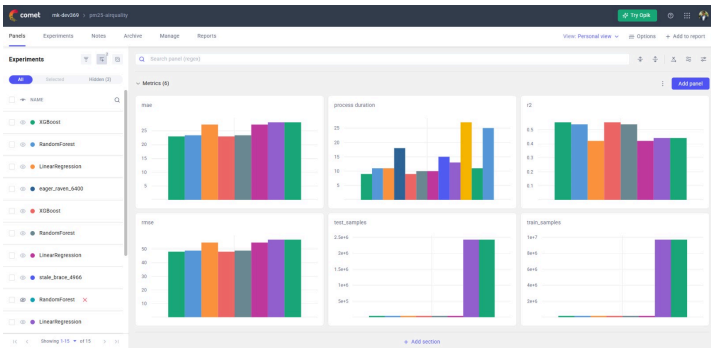


Figure : CometML Experiment Tracking

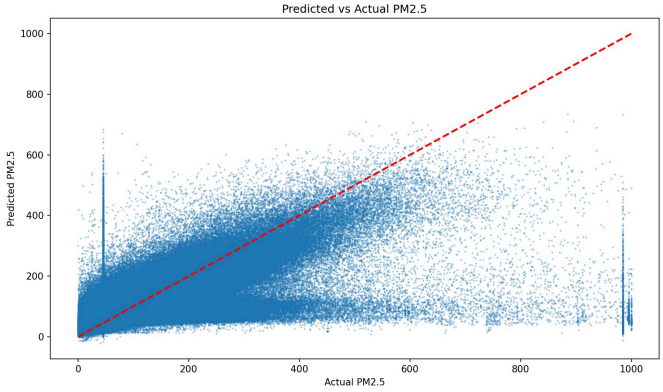
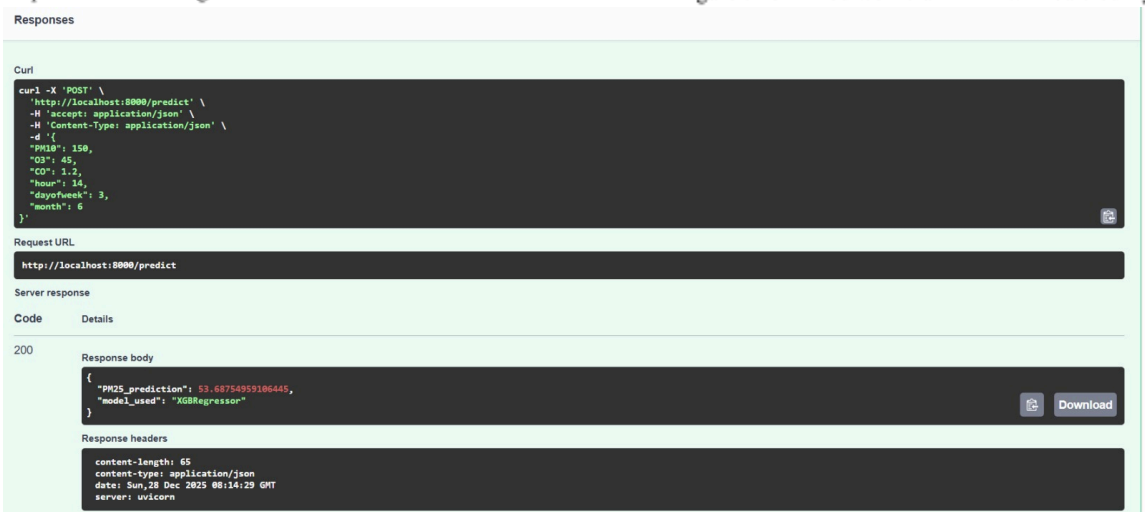
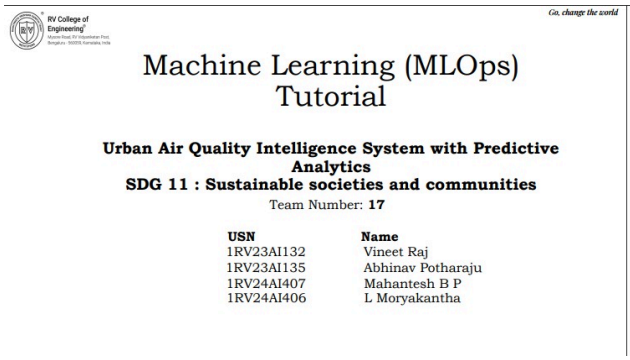


Figure : Estimated v/s Actual PM 2.5 Predicted By Model



CHAPTER 5
SLIDES


RV College of Engineering

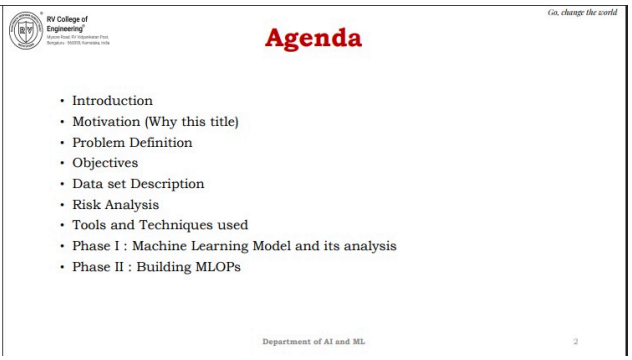
Go, change the world

Machine Learning (MLOps) Tutorial

Urban Air Quality Intelligence System with Predictive Analytics
SDG 11 : Sustainable societies and communities

Team Number: **17**

USN	Name
1RV23AI132	Vineet Raj
1RV23AI135	Abhinav Potharaju
1RV24AI407	Mahantesh B P
1RV24AI406	L Moryakantha

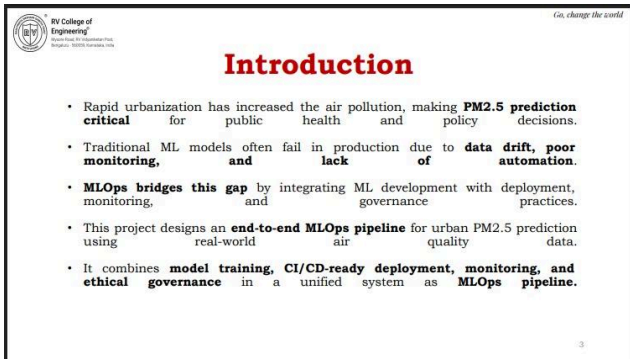

RV College of Engineering

Go, change the world

Agenda

- Introduction
- Motivation (Why this title)
- Problem Definition
- Objectives
- Data set Description
- Risk Analysis
- Tools and Techniques used
- Phase I : Machine Learning Model and its analysis
- Phase II : Building MLOPs

Department of AI and ML 2

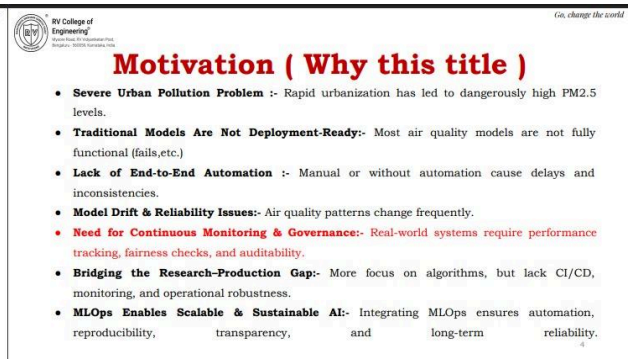

RV College of Engineering

Go, change the world

Introduction

- Rapid urbanization has increased the air pollution, making **PM2.5 prediction critical** for public health and policy decisions.
- Traditional ML models often fail in production due to **data drift, poor monitoring, and lack of automation**.
- **MLOps bridges this gap** by integrating ML development with deployment, monitoring, and governance practices.
- This project designs an **end-to-end MLOps pipeline** for urban PM2.5 prediction using real-world air quality data.
- It combines **model training, CI/CD-ready deployment, monitoring, and ethical governance** in a unified system as **MLOps pipeline.**

3


RV College of Engineering

Go, change the world

Motivation (Why this title)

- **Severe Urban Pollution Problem :-** Rapid urbanization has led to dangerously high PM2.5 levels.
- **Traditional Models Are Not Deployment-Ready:-** Most air quality models are not fully functional (fails,etc.)
- **Lack of End-to-End Automation :-** Manual or without automation cause delays and inconsistencies.
- **Model Drift & Reliability Issues:-** Air quality patterns change frequently.
- **Need for Continuous Monitoring & Governance:-** Real-world systems require performance tracking, fairness checks, and auditability.
- **Bridging the Research–Production Gap:-** More focus on algorithms, but lack CI/CD, monitoring, and operational robustness.
- **MLOps Enables Scalable & Sustainable AI:-** Integrating MLOps ensures automation, reproducibility, transparency, and long-term reliability.

4

RV College of Engineering

Go, change the world

Problem Definition

- Urban areas face **frequent and unpredictable PM2.5 spikes**.
- Traditional air quality systems are **reactive.**
- Existing ML models focus only on **accuracy**, ignoring deployment and monitoring.
- **Data drift, sensor noise, and model decay** reduce long-term reliability.
- Lack of **end-to-end MLOps pipelines (from dev to prod)** for air quality prediction.
- Difficulty in detecting **unusual or hazardous AQI conditions in real time.**

5

RV College of Engineering

Go, change the world

Objectives

- To **predict urban PM2.5 air quality levels** using 3 machine learning models(Linear Regression, Random Forest, XGBoost).
- To **compare 3 ML models** and select the best-performing one.
- To **build an end-to-end MLOps pipeline** which **deploy the trained model as a scalable API ,enable experiment tracking and model versioning, monitor model performance , data drift and system health and ensure governance, fairness, and transparency**

6

RV College of Engineering

Go, change the world

Data-set Description

Dataset 1: Time-Series Air Quality Data of India (2010-2023)

Source: Kaggle

Link: Dataset 2

Description:

- This dataset contains **station-level time-series air quality data** collected across different locations in India from **2010 to 2023**.
- Each CSV file represents measurements from a specific monitoring station.
- The data is recorded at regular time intervals.
- The data has been collected from the **Central Control Room for Air Quality Management**, which indicates that it is a reliable and authoritative source.

Key Features Used:

- Timestamp (Date & Time of measurement)
- PM2.5
- PM10
- O3 (Ozone)
- CO (Carbon Monoxide)

Use in Project:

- This dataset provides **long-term historical pollution trends** and high-frequency measurements.

7

RV College of Engineering

Go, change the world

Data-set Description

Dataset 2: Indian Cities AQI Dataset (2020-2024)

Source: Kaggle

Link: Dataset 2

Description:

- This dataset contains **city-level air quality data** for major Indian cities from **2020 to 2024**.
- It represents aggregated pollution data at the city level rather than individual stations.
- The data has been collected from the Central Pollution Control Board (CPCB)'s repository - CPCB

Key Features Used:

- Timestamp
- PM2.5
- PM10
- O3
- CO

Use in Project:

- This dataset adds **geographical and urban-level pollution patterns**.

8

RV College of Engineering

Go, change the world

Data-set Description

Dataset Integration & Final Dataset

- Both datasets were **cleaned, standardized, and merged** based on **common pollutant columns and timestamp**.
- Column names were normalized (e.g., different date/time formats unified into a single Timestamp column).
- Missing values were handled using **median imputation**.

Final Combined Dataset:

master_airquality_clean.csv

Final Columns Used for Modeling:

- Timestamp
- PM2.5 (Target variable)
- PM10
- O3
- CO
- hour, dayofweek, month (engineered time features)
- Source

9

RV College of Engineering

Go, change the world

Risk Analysis

There are 10 risks which we had discovered and assessed them with score out of 25 (5x5) in which first metric is **Probability** and then second is **Impact** Assessment.

Notations : Rx (Risk x : x belong to 1 -10)

R1. Incomplete or Poor-Quality Environmental Data. (4/5 x 4/5 =16/25)

R2. Model Underperformance due to data drift and bias.(3/5 x 4/5 =12/25)

R3. Integration Issues among MLOps Tools.(3/5 x 3/5 =9/25)

R4. Model Reproducibility failure due to missing environment versions.(2/5 x4/5 =8/25)

R5. Container deployment failure or API downtime.(3/5 x 3/5 =9/25)

R6. Misinterpretation of Air Quality Forecasts (Stakeholder misunderstanding).(2/5 x 4/5 =10/25)

R7. Loss of model or data versions due to repository issues.(1/5 x 4/5 =4/25)

R8. Lack of Model Generalization to New Cities or Unseen Locations.(4/5 x 3/5 =12/25)

R9. Violating Data Usage Rights (Non-open Datasets).(1/5 x 5/5 =5/25)

R10. Inadequate monitoring or failure to detect Performance Degradation.(3/5 x 4/5 =12/25)

10

RV College of Engineering

Go, change the world

Tools and Techniques used

Tools

- **Python** – Core programming language for data processing and modeling
- **Pandas & NumPy** – Data cleaning, preprocessing, and analysis
- **Scikit-learn** – Model training, evaluation, and preprocessing pipelines
- **XGBoost** – High-performance regression model for PM2.5 prediction
- **FastAPI** – REST API for real-time model inference
- **Docker & Docker Compose** – Containerized deployment and environment consistency
- **Prometheus** – Monitoring API performance and system metrics
- **Grafana** – Dashboard
- **Weights & Biases (W&B) , CometML** – Experiment tracking, model versioning, and logging
- **AIF360** – Fairness, bias, and governance analysis
- **KubeFlow**– for pipeline orchestration

Techniques

- Data cleaning, normalization, and feature engineering
- Time-based feature extraction (hour, day, month)
- Regression modeling and model comparison
- Hyperparameter tuning and performance evaluation (RMSE, R^2, MAE)
- Anomaly detection using residual analysis
- Model explainability and governance practices
- Containerized deployment and monitoring (MLOps lifecycle)

11

RV College of Engineering

Go, change the world

Phase I – ML Life Cycle

Exploratory Data Analysis

- Analyzed historical air quality data to understand pollution trends and patterns
- Examined distribution of PM2.5 and other pollutants across time and locations
- Identified missing values, outliers, and inconsistencies in sensor readings
- Studied temporal patterns (hourly, daily, monthly variations in pollution)
- Analyzed correlations between PM2.5 and pollutants like PM10, CO, O_3
- Compared pollution levels across cities and industrial monitoring stations
- Visualized data using SHAP summary plots for interpretability, residual plots and histograms for error analysis, and bar charts for model performance comparison.
- Insights from EDA guided feature engineering and model selection

Department of AI and ML 12

RV College of Engineering

Go, change the world

Phase I – ML Life Cycle

List of algorithms chosen to build ML model

Linear Regression

- Used as a **baseline model**
- Helps understand linear relationships between pollutants and PM2.5

Random Forest Regressor

- Handles **non-linear relationships**
- Robust to noise and overfitting
- Performs well on tabular environmental data

XGBoost Regressor

- Gradient boosting–based ensemble model
- Captures complex interactions between features
- Provides **high accuracy and generalization**
- Selected as the **final best-performing model**

Department of AI and ML 13

RV College of Engineering

Go, change the world

Phase I – ML Life Cycle

Metrics used to evaluate the ML model

RMSE (Root Mean Squared Error)

- Primary evaluation metric
- Penalizes large prediction errors
- Suitable for continuous PM2.5 prediction

R^2 Score (Coefficient of Determination)

- Measures how well the model explains variance in PM2.5
- Helps compare model performance

Model-wise RMSE Comparison

- Linear Regression – baseline performance
- Random Forest – captures non-linear patterns
- XGBoost – best generalization with lowest RMSE

Department of AI and ML 14

Phase I – ML Life Cycle

Evaluation of ML model

1. Models evaluated on **unseen test data** to ensure generalization
2. **RMSE (Root Mean Square Error)** used as primary metric
   a. Penalizes large prediction errors
   b. Suitable for continuous PM2.5 prediction
3. **Linear Regression**
   a. Baseline model
   b. Higher error due to linear assumptions
4. **Random Forest**
   a. Captures non-linear relationships
   b. Better performance than Linear Regression
5. **XGBoost**
   a. Lowest RMSE among all models
   b. Handles complex patterns and feature interactions efficiently
6. **Best model selected based on minimum RMSE**
7. Model performance logged and compared using **Weights & Biases**
8. Final best model saved and versioned for deployment

Phase I – ML Life Cycle

Identification of Best Model

- Three models were trained: **Linear Regression, Random Forest, and XGBoost.**
- All models were evaluated using **RMSE (Root Mean Square Error).**
- **Lower RMSE means better prediction accuracy.**
- Linear Regression showed **high error** and could not capture complex patterns
- Random Forest performed better but showed **slight overfitting**
- XGBoost achieved the **lowest RMSE among all models**
- XGBoost handled **non-linear relationships and feature interactions** well
- It showed **stable performance on unseen test data**
- **XGBoost was selected as the final and best-performing model**

Phase II – MLOPs Life Cycle

	Component	MLOPs Tools
1	Data Management	Grafana ,Git (versioning)
2	Experiment Tracking and Version Control	Weights & Biases (W&B) , Git ,CometML
3	Automation and Pipeline Orchestration	Kubeflow , Docker
4	Model Deployment & Serving	Docker , Fast+API ,Github Actions
5	Monitoring & Governance and collaboration	AIFairness 360 , Prometheus

Phase II – MLOPs Life Cycle

Data Management

Data Collection

- Air quality data collected from **cities and monitoring stations from CPCB** (CSV files).
- Data includes PM2.5, PM10, gases, weather, and timestamps.

Data Cleaning (Using Pandas)

- Handle missing values (mean/median imputation).
- Remove duplicate rows and invalid sensor readings.
- Convert timestamps into usable time features (hour, day, month).

Data Transformation

- Feature scaling (normalization/standardization).
- Feature engineering like lag values and rolling averages.

Data Versioning (Using Git)

- Raw, cleaned, and processed datasets are tracked in Git.
- Ensures reproducibility and rollback if data changes.

Visualization Support

- Grafana dashboards can visualize stored metrics (later stage).

Phase II – MLOPs Life Cycle

Experiment Tracking and Version Control

Initialize Experiment Tracking

- Login to W&B and initialize a run before model training.

Log Hyperparameters

- Learning rate, depth, number of trees, etc. are logged.

Train Multiple Models

- Linear Regression
- Random Forest
- XGBoost

Log Metrics

- RMSE values for each model logged automatically.
- Comparison between models done visually in W&B dashboard.

Model Versioning

- Best model saved as a W&B artifact.
- Enables rollback to older models if needed.

Code Version Control

- Git tracks training scripts and notebooks.

Phase II – MLOPs Life Cycle

Automation & Pipeline Orchestration

Containerization (Docker)

- Training and inference environments packed into Docker images.
- Ensures consistency across systems.

Pipeline Definition (Kubeflow)

- Define pipeline steps:
  - Data preprocessing
  - Model training
  - Evaluation
  - Model storage

Pipeline Execution

- Kubeflow executes steps automatically.
- Handles dependencies between steps.

Scalability

- Pipelines can run on multiple nodes for large datasets.

Phase II – MLOPs Life Cycle

Model Deployment & Serving

Model Export

- Best model saved as `.pkl` or `.joblib`.

API Creation (FastAPI)

- `/predict` endpoint created for PM2.5 prediction.
- Input features passed as JSON.

Model Serving

- Model loaded inside FastAPI application.
- Predictions returned in real time.

Application Server

- Uvicorn used to run the FastAPI app.

Containerized Deployment

- API wrapped in Docker container.

CI/CD (GitHub Actions)

- On code push:
  - Build Docker image
  - Run tests
  - Deploy updated API automatically

Phase II – MLOPs Life Cycle

Monitoring, Governance & Collaboration

API Monitoring (Prometheus)

- Tracks request count, latency, and errors.
- `/metrics` endpoint exposed in FastAPI.

Model Performance Monitoring

- Monitor prediction distribution over time.
- Detect sudden changes (concept drift).

Fairness & Bias Analysis (AIF360)

- Check if predictions are biased across regions or time.
- Generate fairness metrics and reports.

Governance

- Maintain:
  - Model cards
  - Data cards
  - Audit logs

Collaboration

- Teams collaborate using:
  - Git for code
  - W&B dashboards for experiments
  - Shared monitoring dashboards

CHAPTER 6

6.1 Revisiting Model Performance

What We Do

After deployment and during subsequent evaluation cycles, the model's performance is revisited to ensure that prediction quality remains consistent. Key regression metrics such as **Root Mean Square Error (RMSE)**, **Mean Absolute Error (MAE)**, and **R^2 score** are recalculated using updated validation and test datasets. These results are compared with initial training metrics to detect signs of overfitting or underfitting.

Additionally, predicted PM2.5 values are compared with real-world observed values to validate the practical reliability of the model under varying pollution conditions, including peak pollution periods.

Why It Is Important

Air quality data is highly dynamic and affected by seasonal variation, policy changes, and environmental factors. Over time, these changes can cause **model drift**, leading to degraded prediction accuracy. Regular performance reassessment helps identify such degradation early and ensures that the deployed model remains suitable for real-world usage and large-scale deployment.

In Our Project

In our implementation, the **XGBoost model** consistently demonstrated stable performance across training, validation, and test datasets. It achieved a **lower RMSE and MAE** compared to Linear Regression and Random Forest models, indicating superior predictive accuracy and better generalization. The consistency between validation and test results suggested minimal overfitting.

6.2 Hyperparameter Tuning and Performance Optimization

Common Hyperparameters Tuned

To optimize model performance, several hyperparameters were adjusted, including:

- Number of trees (**n_estimators**)
- Learning rate
- Maximum tree depth
- Subsample and column sampling ratios

These parameters directly influence model complexity, learning behavior, and resistance to overfitting.

Techniques Used

Hyperparameter tuning was carried out using:

- Manual experimentation guided by domain knowledge
- **Grid Search** and **Random Search** strategies
- Validation-based comparison of RMSE and MAE metrics

Each experiment was logged and tracked to ensure reproducibility and systematic comparison.

Figure :GPU Utilization of the system

Figure :Power usage of the system

Impact on Performance

Effective tuning reduced prediction errors, improved robustness on unseen data, and enhanced the model's ability to handle extreme pollution values. Optimized configurations also helped balance model complexity and inference efficiency.

In Our Project

The optimized XGBoost configuration resulted in a noticeable reduction in RMSE and improved generalization. The tuned model showed better stability during high pollution episodes, preventing overfitting to extreme values while maintaining accuracy during normal conditions.

6.3 Interpretability

Why It Matters

Interpretability is essential in environmental and public health applications. Transparent models build stakeholder trust, support governance and audit requirements, and enable effective debugging when predictions appear incorrect or unexpected.

Techniques Used

The following interpretability techniques were applied:

- **Feature importance analysis** from XGBoost
- **SHAP (SHapley Additive exPlanations)** for both global and local explanations
- Error distribution and residual analysis to identify systematic biases

In Our Project

Analysis revealed that **PM10, CO, O_3**, and time-based features such as hour and season had the strongest influence on PM2.5 predictions. SHAP visualizations clearly explained pollution spikes and helped validate that the model relied on meaningful environmental signals, improving transparency and confidence in predictions.

6.4 Model Versioning and Registry

Why It Is Needed

Model versioning is critical for managing multiple model iterations, enabling rollback to previous versions, and ensuring reproducibility. It also allows objective comparison between models trained with different data or configurations.

Tools Used

- **Git** for source code versioning
- **Weights & Biases (W&B)** for experiment tracking and artifact management
- A structured **model registry** to organize model versions and metadata

In Our Project

The best-performing model was saved as **best_pm25_model.pkl** and registered as a W&B artifact with complete metadata, including training dataset version, hyperparameters, and evaluation metrics. Each model version was uniquely identifiable and traceable to its training context.

6.5 Documentation of Model Updates

What Is Documented

For every model update, the following information was documented:

- Dataset and feature versions
- Hyperparameter configurations
- Training and evaluation metrics
- Deployment date and model version
- Known limitations and assumptions

Why It Is Important

Comprehensive documentation ensures transparency, audit readiness, and compliance with governance standards. It enables future teams to understand model decisions and reproduce results if needed.

In Our Project

Model cards and governance reports were created to summarize model behavior, performance characteristics, and limitations. Fairness and bias evaluations were documented, and a complete version history was maintained to support audits and long-term maintenance.

CHAPTER 7

7.1 Overview of CI/CD for Machine Learning

CI/CD stands for **Continuous Integration and Continuous Deployment**, a practice originally designed for automating software build, test, and release processes. In traditional software systems, CI/CD focuses primarily on validating source code changes and deploying updated applications.

In **Machine Learning systems**, CI/CD extends beyond code to include **data, models, configurations, and environments**. Any change in training data, feature engineering logic, hyperparameters, or model architecture can significantly affect model performance. Therefore, ML CI/CD pipelines are designed to automatically retrain models, validate performance against defined thresholds, and deploy updated models in a controlled manner.

CI/CD automation ensures:

- Faster iteration and experimentation
- Reduced manual intervention and human error
- Reproducibility across model versions
- Reliable deployment of validated models

For air quality prediction systems, CI/CD is particularly important because **model accuracy can degrade over time due to data drift and seasonal variation**. Continuous pipelines allow the system to adapt quickly and remain production-ready.

7.2 Tools Setup: GitHub Actions / Jenkins / GitLab CI

Several CI/CD tools are available for automating machine learning workflows, including **GitHub Actions**, **Jenkins**, and **GitLab CI**. In this project, **GitHub Actions** was selected due to its ease of use, YAML-based configuration, and seamless integration with GitHub repositories.

GitHub Actions

GitHub Actions enables developers to define CI/CD workflows using YAML files stored within the repository. These workflows can be triggered by:

- Code pushes
- Pull requests
- Scheduled cron jobs
- Manual triggers

GitHub Actions supports automation of dependency installation, model training, evaluation, Docker image creation, and deployment. Its native integration makes it well-suited for **student projects and small-to-medium MLOps systems**, while still following industry-standard practices.

7.3 Automating Model Training and Testing

Automated Training

The CI pipeline automates the end-to-end training process:

1. Pulls the latest code and configuration files
2. Installs required Python dependencies
3. Executes the training script (`train.py`)
4. Logs hyperparameters and metrics to **Weights & Biases**
5. Stores the trained model as a versioned artifact

This ensures consistent training environments and repeatable experiments.

Automated Testing and Quality Gates

Automated testing is integrated to prevent poor-quality models from reaching deployment. Tests include:

- Data validation checks (missing values, schema mismatch)
- Model performance validation using RMSE thresholds
- Inference sanity checks using sample inputs
- API unit tests for prediction endpoints

A **quality gate** is enforced:
If the model fails to meet predefined performance criteria, the pipeline stops automatically and deployment is blocked.

7.4 Building and Running a Deployment Pipeline

Deployment Flow

When code changes pass all validation steps, the deployment pipeline is triggered. A Docker image containing the trained model and application code is built. The model is served through a **FastAPI** application, and the API is launched using **Uvicorn**.

The containerized application is then deployed to the target environment.

Key Components

- **Docker:** Ensures environment consistency across development and production
- **FastAPI:** Exposes REST endpoints for predictions
- **Uvicorn:** Runs the application server
- **Prometheus:** Collects application and system metrics

Deployment Strategies

The system supports:

- Local Docker-based deployment
- Cloud-based deployment on virtual machines
- Design readiness for future Kubernetes-based orchestration

Rollback Mechanism

If deployment fails or performance degrades:

- The pipeline supports rollback to a previous stable model
- Docker images are versioned for safe recovery
- Model artifacts stored in W&B enable quick restoration

7.5 Continuous Integration Hands-On Exercise

Practical CI Workflow Example

When a developer modifies model code or configuration files and pushes the changes to GitHub, the CI/CD pipeline is automatically triggered.

Workflow Steps

1. Code checkout and dependency installation
2. Data and unit validation tests
3. Model training and evaluation
4. Metric logging to Weights & Biases
5. Docker image build
6. Deployment upon successful validation

If any step fails, the pipeline terminates and notifies the developer.

Outcome

This automated workflow:

- Accelerates experimentation cycles
- Eliminates manual deployment errors
- Improves reliability and traceability
- Ensures only validated models are deployed

7.6 Scheduling and Continuous Retraining

In addition to event-based triggers, scheduled workflows are configured to:

- Periodically retrain models using recent data
- Re-evaluate performance metrics
- Detect performance degradation proactively

Scheduled retraining ensures the system adapts to long-term changes in air quality patterns.

7.7 Security and Access Control

To ensure secure operations:

- Secrets such as API keys are stored securely using GitHub Secrets
- Access to deployment workflows is restricted
- Only approved branches can trigger production deployments

‘

CHAPTER 8

8.1 Model Monitoring Concepts and Key Performance Indicators (KPIs)

Why Monitoring Is Needed

In real-world deployments, machine learning models operate in dynamic environments where data distributions and external conditions continuously evolve. In air quality prediction systems, changes in traffic patterns, weather conditions, seasonal events, or sensor recalibration can silently degrade model accuracy. Such degradation may go unnoticed without proper monitoring, leading to unreliable predictions that can negatively affect public health decisions.

Continuous monitoring ensures early detection of performance issues, enables timely corrective actions, and maintains trust in the deployed system.

Key Performance Indicators (KPIs)

The following KPIs are continuously monitored:

- **Model Performance Metrics:** RMSE, MAE, and prediction error trends
- **Data Drift Metrics:** Changes in input feature distributions over time
- **Concept Drift Metrics:** Degradation in prediction accuracy and error trends
- **System Metrics:** API latency, error rate, throughput, and uptime

8.2 Tools Used: Prometheus and Grafana

Prometheus

Prometheus is an open-source monitoring system used to collect real-time metrics from the deployed FastAPI service. It scrapes metrics exposed by the application and stores them as time-series data.

In this project, Prometheus was used to monitor:

- API request count and throughput
- Prediction latency
- HTTP error rates

- System resource utilization

These metrics help assess the operational stability of the deployed model.

Grafana

Grafana is a visualization and analytics platform integrated with Prometheus. It was used to create interactive dashboards that display system and model metrics in real time.

Grafana dashboards enabled:

- Visualization of API performance trends
- Identification of latency spikes and error patterns
- Preparation of alert-ready views for rapid response

8.3 Detecting Concept Drift and Data Drift

Data Drift

Data drift occurs when the statistical distribution of input features changes over time.

Examples:

- Sudden increase in PM2.5 levels during festival seasons
- Sensor recalibration altering measurement ranges

Detection Methods:

- Kolmogorov–Smirnov (KS) test
- Population Stability Index (PSI)
- Drift reports generated using **EvidentlyAI**

These methods compare recent input data with reference training data to identify significant distribution shifts.

Concept Drift

Concept drift occurs when the relationship between input features and the target variable (PM2.5) changes.

Examples:

- Identical PM10 levels producing higher PM2.5 due to changes in weather conditions
- Policy or environmental changes affecting pollution dynamics

Detection Methods:

- Monitoring RMSE and MAE trends over time
- Comparing prediction errors across recent time windows
- Performance degradation alerts when error exceeds predefined thresholds

8.4 Model Retraining and Redeployment

When Retraining Is Required

Model retraining is triggered under the following conditions:

- Data or concept drift exceeds defined thresholds
- RMSE or MAE crosses acceptable limits
- New or updated data sources are introduced

Retraining Workflow

1. Recent production data is collected and merged with historical data
2. The automated training pipeline is executed

3. Model performance is evaluated against acceptance criteria
4. The new model is reviewed and approved

Redeployment Strategy

Once approved, the updated model is:

- Packaged using Docker
- Deployed through the CI/CD pipeline
- Logged as a new model version

The previous model version is archived to support rollback if necessary.

8.5 Logging, Alerting, and Dashboard Setup

Logging

Structured logging is implemented to capture:

- Prediction inputs and outputs
- Model version identifiers
- System errors and exceptions

These logs support traceability, debugging, and audit requirements.

Alerting

Alerts are configured to notify stakeholders when:

- API latency exceeds defined thresholds
- Data or concept drift is detected
- Model performance drops below acceptable limits

Prometheus Alertmanager and **Grafana alert rules** are used to trigger notifications.

Dashboard Setup

Dashboards display:

- Model performance metrics
- API health and availability
- Drift trends and historical comparisons
- Traffic volume and usage patterns

CHAPTER 9

9.1 Post-Deployment Performance Review

After deployment, the model's performance was continuously reviewed to assess its behavior in real-world conditions. Live prediction metrics such as **RMSE and MAE** were compared with offline validation results to identify deviations in accuracy. This comparison helped verify whether the model generalized well beyond the training environment.

In addition to numerical metrics, **prediction stability** was analyzed by observing output trends over time. Sudden spikes, systematic bias, or prolonged error increases were flagged for investigation. This review enabled early detection of performance degradation caused by environmental changes or data drift.

The post-deployment review confirmed that the deployed model maintained consistent performance while also highlighting opportunities for further optimization.

9.2 Continuous Improvement via Feedback Loops

Continuous improvement is achieved through **closed feedback loops** integrated into the MLOps pipeline. Feedback is collected from multiple sources, including:

- Newly ingested production data
- Monitoring alerts related to drift or performance degradation
- Historical prediction errors and residual analysis

This feedback is periodically incorporated into the training dataset, enabling the model to adapt to evolving air quality patterns. Automated retraining ensures improvements are systematically evaluated before deployment, reducing the risk of regressions.

This iterative learning approach ensures that the system evolves continuously rather than remaining static.

9.3 Governance and Ethical AI Practices

Given the public health implications of air quality prediction, strong governance and ethical AI practices are essential. Bias and fairness analysis was performed using **AIF360**, focusing on regional and temporal fairness in predictions.

The governance framework ensured that:

- Prediction errors were not disproportionately higher for specific regions
- Temporal bias across seasons or special events was minimized
- Model decisions were explainable and transparent

Ethical safeguards were embedded throughout the pipeline to promote accountability, fairness, and responsible AI usage.

9.4 Documentation: Model Cards and Data Cards

Model Cards

Model Cards were created to provide standardized documentation of:

- Model objectives and intended use
- Training datasets and evaluation metrics
- Interpretability and explainability techniques
- Known limitations, assumptions, and risks

These cards help stakeholders understand appropriate usage and avoid misinterpretation of predictions.

Data Cards

Data Cards documented:

- Data sources and collection methods
- Temporal and geographic coverage

- Licensing and usage constraints
- Potential data biases and quality issues

Together, Model Cards and Data Cards support transparency, traceability, and ethical compliance.

9.5 Audit and Review Checklist

A structured audit and review checklist was followed to ensure operational readiness and compliance. The checklist verified:

- Data lineage and dataset versioning
- Model versioning and deployment history
- Monitoring, logging, and alerting configuration
- Compliance with governance and ethical standards

This process ensured that the system is **audit-ready**, maintainable, and suitable for long-term operation.

9.6 Lessons Learned

During the project, several key lessons were identified:

- Data quality has a greater impact on model performance than model complexity.
- Monitoring and drift detection are critical for maintaining accuracy post-deployment.
- Automation significantly reduces operational overhead and human error.
- Interpretability is essential for trust in environmental ML systems.

These lessons provide valuable guidance for future MLOps projects.

CHAPTER 10

10.1 End-to-End MLOps Pipeline Demonstration

The complete end-to-end MLOps pipeline was demonstrated, showcasing automated workflows from **data ingestion** through **training, evaluation, deployment, monitoring, and retraining**. The demonstration illustrated how changes in data or model configuration automatically triggered the CI/CD pipeline.

This validated the effectiveness of automation and confirmed that the system could operate with minimal manual intervention.

10.2 Final Model Evaluation and Results

After comprehensive evaluation, **XGBoost** was selected as the final production model due to its superior performance across multiple metrics. It consistently achieved:

- Lower RMSE and MAE compared to baseline models
- Stable generalization on unseen data
- High accuracy in AQI classification tasks

These results confirmed that XGBoost effectively balances accuracy, robustness, and operational efficiency.

10.3 Project Documentation and Reporting

Extensive technical documentation was produced to support reproducibility and long-term maintenance. The documentation includes:

- System architecture and workflow diagrams
- Detailed risk analysis and mitigation strategies
- CI/CD pipeline configurations

- Monitoring and alerting setup
- Governance and ethical AI practices

This documentation ensures knowledge transfer and future extensibility.

10.4 Final Presentation and Evaluation

The project was presented through a structured technical presentation and live demonstration. The evaluation emphasized:

- Depth of MLOps implementation
- Automation and monitoring maturity
- Governance and ethical considerations
- Practical relevance to real-world urban environments

The successful evaluation demonstrated the project's readiness for real-world deployment.

10.5 Limitations of the System

Despite strong performance, the system has certain limitations:

- Dependence on historical data quality and sensor reliability
- Limited inclusion of meteorological variables due to missing data
- Model performance may vary under extreme or unseen environmental conditions

Recognizing these limitations helps set realistic expectations.

10.6 Future Scope

Future enhancements may include:

- Integration of real-time IoT sensor streams
- Inclusion of advanced deep learning models
- Deployment on Kubernetes for large-scale scalability
- Enhanced fairness monitoring across socio-economic regions
- Integration with public alert and decision-support systems